

Problem Set 6

CMSC 27200: Theory of Algorithms, Winter 2025

Assigned February 25, Due March 7

Note: There will not be resubmissions for this problem set.

Problem 0: Collaborators and Outside Sources

Who did you collaborate with?

Did you use any outside sources? If so, which sources did you use?

Problem 1

For each of the following yes/no problems, either show that the problem is in NP or explain why we should believe that it is not in NP. Hint is available.

- (a) Fixed-length spanning tree: Given a number L and a connected undirected graph G on n vertices where each edge $e = \{u, v\}$ has a length l_e , is there a spanning tree T such that the total length of T is L (i.e., $\sum_{e \in T} l_e = L$)?
- (b) Longest Increasing Subsequence: Given a sequence of distinct numbers $x_1, \dots, x_n$ and a natural number $k \in [n]$, is the length of the longest increasing subsequence of numbers equal to k ?

For example, if we have the sequence of numbers 8, 5, 9, 2, 6, 3, 4 and $k = 2$ then the answer would be no because the longest increasing subsequence is 2, 3, 4 which has length 3.
- (c) Vertex cover size: Recall that a vertex cover of a graph G is a set of vertices V such that for every edge $e = \{u, v\}$, either $u \in V$ or $v \in V$ (or both). Given a graph G , is the minimum size of a vertex cover of G equal to k ?
- (d) Hermits: Given an undirected graph G and positive integers k and l , is it possible to place hermits on k different vertices of G so that the distance between any two hermits is at least l ?

Note: For parts b, c, and d, n^k is not polynomial time as k may depend on n .

Note: For each of these problems which you believe is not in NP, you should explain why you believe the problem is hard and why it is difficult to verify a yes answer.

Remark 0.1. *One way to give strong evidence that a problem is not in NP is to show that the problem is co-NP-hard. You do not need to do this, though you are welcome to do so.*

Problem 2: Balanced Teams

Let's say that you are in charge of splitting $2n$ players with skill levels $x_1, \dots, x_{2n} \in [S]$ into two teams. Can you divide the players into two teams of size n such that the total skill level of each team is the same? In other words, can you find $A, B \subseteq [2n]$ such that $|A| = |B| = n$, $A \cap B = \emptyset$, and $\sum_{i \in A} x_i = \sum_{j \in B} x_j$?

Give a $\text{poly}(n, S)$ time algorithm to solve this problem, explain why your algorithm is correct, and analyze its runtime.

Problem 3: Job Distribution

You are organizing a big convention and there is plenty of work to do. There are a total of m jobs which need to be done, each of which is of one of k types. Fortunately, you have n volunteers, but each volunteer is only willing to do a certain number of each type of job (doing the same thing over and over again gets boring) and a certain number of jobs in total. How can you assign your volunteers to the jobs to satisfy all of these constraints or show that there is no way to do so?

Example 0.2. *Consider the case when you have three volunteers, Margaret, Nancy, and Owen, there are 10 jobs of type 1, 15 jobs of type 2, and 20 jobs of type 3, and your volunteers' willingness to work is as follows:*

1. *Margaret is willing to do 8 jobs of type 1, 10 jobs of type 2, 5 jobs of type 3, and a maximum of 15 jobs overall.*
2. *Nancy is willing to do 3 jobs of type 1, 3 jobs of type 2, 12 jobs of type 3, and a maximum of 16 jobs overall.*
3. *Owen is willing to do 1 job of type 1, 3 jobs of type 2, 14 jobs of type 3, and a maximum of 16 jobs overall.*

If so, we can complete all of the jobs by having Margaret do 6 jobs of type 1 and 9 jobs of type 2, having Nancy do 3 jobs of type 1, 3 jobs of type 2, and 9 jobs of type 3, and having Owen do 1 job of type 1, 3 jobs of type 2, and 11 jobs of type 3.

We can express this problem mathematically as follows. We are given non-negative integers $\{a_i : i \in [k]\}$, $\{b_{ij} : i \in [k], j \in [n]\}$, and $\{c_j : j \in [n]\}$ where a_i is the number of jobs of type i that need to be done, b_{ij} is the maximum number of times volunteer j is willing to do a job of type i , and c_j is the maximum number of jobs volunteer j is willing to do. We want to find non-negative integers $\{x_{ij} : i \in [k], j \in [n]\}$ such that:

1. For all $i \in [k]$, $\sum_{j=1}^n x_{ij} = a_i$ (all of the jobs get done).
2. For all $i \in [k]$ and $j \in [n]$, $x_{ij} \leq b_{ij}$ (no volunteer does too many jobs of a given type).

3. For all $j \in [n]$, $\sum_{i=1}^k x_{ij} \leq c_j$ (no volunteer does too many jobs).

Give a $\text{poly}(k, n, m)$ time algorithm to solve this problem (where $m = \sum_{i=1}^k a_i$ is the total number of jobs which need to be done), prove that your algorithm is correct, and analyze its runtime.

Extra Credit Problem: Bundles

Let's say that you are an artist selling n paintings $p_1, \dots, p_n$. You receive orders from m customers. However, most of your customers do not just want one painting. Instead, each customer j wants a subset $S_j \subseteq \{p_1, \dots, p_n\}$ of your paintings. If they cannot buy all of the paintings in their bundle S_j then they will not buy any paintings. Is it possible to satisfy at least k of the customers?

We can express this problem mathematically as follows. Given subsets $S_1, \dots, S_m \subseteq [n]$, is there a set $I \subseteq [m]$ such that $|I| = k$ and for all distinct $i, j \in I$, $S_i \cap S_j = \emptyset$ (no painting is sold more than once)?

Prove that this problem is NP-complete. Hint is available.

Hints

Extra credit problem: Reducing independent set to this problem will work well. How can you construct bundles corresponding to each vertex so that two bundles clash if and only if the vertices are adjacent to each other?

Note: Don't forget to briefly show that this problem is in NP.